

GlucoFlow

An End-to-End Cloud-Native Data Architecture

for Continuous Glucose Monitoring (CGM) Remote Patient Monitoring

Project Specification & One-Week Build Plan

Document Status: Draft for Review

Version: 0.1

Date: 2026-10-04

Purpose: Workshop + Data Architecture Learning
Project

Contents

1	Executive Summary	2
1.1	Primary Goals	2
2	Business Context	2
2.1	Hypothetical Client	2
2.2	The Problem	2
2.3	Value Proposition	2
2.4	Clinical Anchors (Published Standards)	3
3	Data Sources	3
4	Architecture Overview	4
4.1	Logical Flow	4
5	Requirements Coverage Matrix	4
6	Seven Traits of a Modern Data Architecture	5
7	Data Model (Summary)	6
7.1	Relational (RDS PostgreSQL)	6
7.2	Hierarchical (S3 JSON / Postgres JSONB)	6
7.3	Schemaless (DynamoDB)	6
7.4	Analytical / Temporal (S3 + Apache Iceberg)	6
8	Validation & Cleansing Rules	6
9	One-Week Build Schedule	7
10	Risks & Mitigations	8
11	Indicative Cost Posture	9
12	Consolidated Open Questions for Your Review	9
13	Glossary	9

1 Executive Summary

GlucoFlow is a cloud-native data architecture project built around **Continuous Glucose Monitoring (CGM)** data for a hypothetical Remote Patient Monitoring (RPM) program. The project is intentionally designed as a teaching and presentation vehicle: it exercises every stage of a modern data lifecycle — **Collect, Store, Analyze, and Access** — while demonstrating the seven traits of a modern data architecture.

Why CGM?

CGM data is time-series, high-volume, clinically meaningful, and comes with published standards (the 2019 International Consensus on Time in Range). This gives us *real data*, *real validation rules*, and a *real business case* — rather than invented numbers.

The project ingests real patient data (Shanghai T1DM/T2DM open dataset) and validated synthetic data (**simglucose**), processes it through both streaming and batch pipelines on AWS, stores it across relational, hierarchical, and schemaless models, and exposes insights through APIs and an AI agent layer connected via the Model Context Protocol (MCP).

1.1 Primary Goals

1. Demonstrate a complete, defensible data architecture end-to-end.
2. Integrate AWS cloud services (supports the parallel AWS workshop).
3. Teach data organization: ingestion, validation, modeling, and serving.
4. Produce a presentable artifact with a credible consulting narrative.

2 Business Context

2.1 Hypothetical Client

A health system or digital-health company running a Remote Patient Monitoring program for diabetes patients.

2.2 The Problem

- Each patient generates hundreds of readings per day; clinicians review and document these manually.
- CGM data is stuck in vendor portals rather than flowing into the EHR (an interoperability gap).
- In the US, CGM interpretation and RPM services carry billing codes requiring documented review, making report quality and turnaround a revenue issue. (*To be confirmed during research.*)

2.3 Value Proposition

Client Pain	Pipeline Capability	Business Outcome
Clinician time on raw data	Automated metrics & summary drafts	Lower review time per patient

Client Pain	Pipeline Capability	Business Outcome
Inconsistent reports	Standardized metrics (2019 Consensus)	Consistent quality
Data stuck in vendor silos	FHIR output	EHR integration
Compliance risk	De-identification & audit logging	HIPAA-aware design

2.4 Clinical Anchors (Published Standards)

2019 International Consensus on Time in Range

- **Time in Range** (70–180 mg/dL): >70% of readings
- **Time Below Range**: <4% below 70; <1% below 54
- **Time Above Range**: <25% above 180; <5% above 250
- **Glucose Variability** (CV): ≤36%
- **Data Sufficiency**: ≥70% of possible readings over 14 days

The data-sufficiency rule directly ties cleansing to clinical validity: if gaps are too large, the report is withheld.

3 Data Sources

Source	Role	Access	Notes
Shanghai T1DM/T2DM (Zhao et al., 2023)	Primary “real client data”	figshare, no approval	15-min sampling → real normalization problem
simglucose (UVA/Padova)	Edge-case stress testing	PyPI	FDA-accepted simulator; models sensor noise
Dexcom Clarity-style export	Realistic “messy” format	Synthetic	“Low”/“High” strings, mixed event rows
Synthea FHIR	Mock EHR / internal integration	Open generator	Patient demographics
USDA FoodData Central	Third-party carb lookup	Public API	Stretch goal

Assumption / Open Question

Please confirm the Shanghai dataset license is acceptable for a work presentation before we rely on it as the primary source.

4 Architecture Overview

The architecture follows the four-stage data lifecycle, implemented on AWS with a medallion (Bronze/Silver/Gold) storage pattern.

4.1 Logical Flow

1. **Collect:** Streaming (Kinesis) + Batch (S3/EventBridge).
2. **Validate & Cleanse:** Lambda / Glue Data Quality; quarantine.
3. **Transform & Stage:** Resample to 5-min grid; compute metrics; Bronze→Silver→Gold.
4. **Store:** RDS (relational), S3/JSONB (hierarchical), DynamoDB (schemaless), Iceberg (temporal).
5. **Analyze:** OLTP (RDS), real-time (Kinesis+Lambda), OLAP (Athena), AI (Bedrock/watsonx).
6. **Access:** API Gateway REST + FHIR output; MCP tools; dashboard.

5 Requirements Coverage Matrix

Tier key: *Live* = built & demoed; *Stretch* = if time allows; *Diagram* = designed, not built.

Requirement	GlucoFlow Coverage	AWS Service(s)	Tier
COLLECT			
Real-time tracking fields	Each record carries <code>event_time</code> , <code>ingest_time</code> , <code>source_system</code> , <code>batch_or_stream</code> , <code>lineage_id</code>	Lambda (stamping)	Live
End-of-day batch	Nightly Dexcom-style CSV exports land in S3 and trigger processing	S3, EventBridge Scheduler	Live
Streaming	Simulated devices send readings every 5 minutes	Kinesis Data Streams, Lambda	Live
Data validation	Schema and range checks; failing records quarantined	Lambda (pydantic), Glue DQ	Live
Cleansing	Handle “Low”/“High”, dedupe, normalize units/timezones, flag gaps	Lambda / Glue	Live
Transformation	Resample to 5-min grid; compute time-in-range metrics	Glue (PySpark) / Lambda	Live
Staging	Bronze → Silver → Gold medallion layers	S3	Live
Loading	Gold data to warehouse, database, and key-value store	Glue, Athena, RDS, DynamoDB	Live
Internal integration	Mock EHR patient records (Synthea FHIR)	S3 → RDS	Live

Requirement	GlucoFlow Coverage	AWS Service(s)	Tier
External integration	Shanghai T1DM/T2DM open dataset	S3	Live
Third-party integration	USDA FoodData Central API (carb lookup)	Lambda	Stretch
MCP	MCP server exposing pipeline query tools to an AI agent	Lambda/container + API	Live
STORE			
Relational	Patients, enrollments, devices, clinician reviews	RDS PostgreSQL	Live
Hierarchical	FHIR bundles; JSON report documents	S3 (JSON), Postgres JSONB	Live
Schemaless	Raw device payloads, latest state, alerts	DynamoDB	Live
Historical/temporal	Iceberg time travel; valid-from/valid-to history	S3+Iceberg, Glue Catalog, Postgres	Live
ANALYZE			
OLTP processing & query	Enrollments, review sign-offs, alert acks	RDS PostgreSQL	Live
Real-time query & analysis	Rolling 1-hour stats; low-glucose alerts in seconds	Kinesis → Lambda → DynamoDB	Live
Historical insights (OLAP)	14-day TIR reports, cohort trends, sufficiency checks	Athena on Iceberg	Live
Leverage AI	Draft clinician summaries; agent answers via MCP tools	Bedrock / watsonx	Live
ACCESS			
APIs	REST API for metrics, reports, alerts; FHIR output	API Gateway + Lambda	Live
Readily accessible	OpenAPI docs, dashboard, MCP tools	API Gateway, Streamlit (local)	Live

Assumption / Open Question

AI layer: do you want **Amazon Bedrock** (keeps everything AWS-native) or **IBM watsonx** (matches the IBM tool from your workshop)? This decision affects the agent/LLM wiring. Please confirm.

6 Seven Traits of a Modern Data Architecture

Trait	How GlucoFlow Demonstrates It	Tier
Cloud-native	Built on managed AWS services (serverless-first)	Live
Robust	Validation, quarantine, retries, dead-letter queues, idempotent writes	Live
Scalable	Kinesis shards + Lambda concurrency + S3/Athena scale independently	Live
Portable	Open formats (Parquet, Iceberg), containerizable MCP server, IaC	Diagram
Pipelines	Separate streaming and batch pipelines, orchestrated and observable	Live
Real-time enabling	Sub-minute alerting path from device to dashboard	Live
Decoupled & extensible	Event-driven stages; add sources/sinks without rework	Live

7 Data Model (Summary)

7.1 Relational (RDS PostgreSQL)

- **patient** — de-identified patient master, with temporal history columns (**valid_from**, **valid_to**).
- **device** — device/transmitter metadata, sampling interval.
- **enrollment** — links patient to RPM program and clinician.
- **clinician_review** — sign-offs, timestamps, documentation (OLTP).

7.2 Hierarchical (S3 JSON / Postgres JSONB)

- FHIR Bundle: Patient → Observation → components.
- Report documents: nested metric blocks per 14-day window.

7.3 Schemaless (DynamoDB)

- **raw_payload** — as-received device JSON (audit & replay).
- **patient_latest_state** — most recent reading + rolling stats.
- **alerts** — active low/high glucose alerts.

7.4 Analytical / Temporal (S3 + Apache Iceberg)

- **glucose_fact** — 5-minute normalized readings, partitioned by date.
- Iceberg snapshots enable *time travel* and reproducible reports.

8 Validation & Cleansing Rules

Rule	Logic	Action on Failure
Range check	Glucose must be 40–400 mg/dL; “Low”/“High” mapped to 39/401 flags	Flag, keep, mark out-of-range

Rule	Logic	Action on Failure
Schema check	Required fields present and typed	Quarantine record
Duplicate check	Same patient + timestamp	Keep first, drop rest
Timezone normalize	All times to UTC + patient local offset stored	Correct in place
Gap detection	Compare actual vs expected readings per 14 days	Compute sufficiency %
Sufficiency gate	<70% readings	Withhold report, raise warning

9 One-Week Build Schedule

How to read this

Each day lists **Tasks**, a **Deliverable**, and a **Done-when** criterion. Days are sized for roughly 5–7 focused hours. Stretch items are only touched if the Live tier for that day is complete.

Day 1 — Foundation & Data

- Download Shanghai dataset; inspect schema; confirm license.
- Install `simglucose`; generate 5 synthetic patients, 14 days.
- Write a Dexcom-style “messy” CSV generator (Low/High, mixed rows).
- Set up AWS account, S3 buckets (bronze/silver/gold/quarantine), IAM.

Deliverable: Raw data in S3 bronze + local sample set.

Done-when: You can list all three source types in the bronze bucket.

Day 2 — Ingestion (Batch + Streaming)

- Build batch path: S3 event → EventBridge → Lambda stamp & land.
- Build streaming path: device simulator → Kinesis → Lambda.
- Add tracking fields (`event_time`, `ingest_time`, `lineage_id`).

Deliverable: Both pipelines writing to Silver with lineage.

Done-when: A simulated reading appears in DynamoDB within 1 minute.

Day 3 — Validation, Cleansing & Transformation

- Implement pydantic/Glue validation; wire quarantine bucket.
- Cleansing: Low/High mapping, dedupe, timezone, unit normalize.
- Resample to 5-minute grid; write `glucose_fact` Iceberg table.

Deliverable: Clean, normalized Gold/Iceberg table.

Done-when: Quarantine catches a deliberately broken record.

Day 4 — Storage Models & Integration

- Load RDS relational tables (patient, device, enrollment, review).
- Load Synthea FHIR bundles (hierarchical) to S3/JSONB.
- Implement temporal history columns + Iceberg time-travel demo.

- (Stretch) USDA FoodData Central carb lookup.

Deliverable: All four storage models populated and queryable.

Done-when: A time-travel query returns a prior snapshot.

Day 5 — Analytics & AI

- Athena queries: 14-day TIR, TBR, TAR, CV, sufficiency.
- Real-time rolling stats + alert logic (Kinesis → Lambda → DynamoDB).
- Build MCP server exposing query tools (get_metrics, get_alerts, get_report).
- Wire AI layer (Bedrock or watsonx) to draft clinician summaries via MCP.

Deliverable: Working metrics + AI-drafted summary for one patient.

Done-when: The agent answers “What is patient X’s time in range?” using live data.

Day 6 — Access Layer & Dashboard

- Build REST API (API Gateway + Lambda): metrics, reports, alerts endpoints.
- Add FHIR-shaped output endpoint for EHR-integration story.
- Publish OpenAPI docs.
- Build a simple Streamlit dashboard (local) consuming the API.

Deliverable: Callable API + dashboard showing one patient end-to-end.

Done-when: The dashboard renders a report pulled entirely through the API.

Day 7 — Hardening, Narrative & Presentation

- End-to-end dry run: stream → alert → report → API → dashboard.
- Add retries / dead-letter queue to show robustness.
- Capture architecture diagram and screenshots.
- Build the presentation deck mapping each requirement to a demo moment.
- Draft the consulting narrative (problem → solution → outcome).

Deliverable: Rehearsed demo + slide deck + diagram.

Done-when: You can run the full demo in under 10 minutes without manual fixes.

Assumption / Open Question

This schedule assumes solo work at 5–7 hrs/day. If you have less time, drop the Stretch items (USDA API, portability IaC) first, then collapse the dashboard (Day 6) into a Post-man/OpenAPI demo.

10 Risks & Mitigations

Risk	Impact	Mitigation
AWS cost overrun	Unexpected charges	Use free-tier, serverless, teardown script, budget alarm
Scope too large for 7 days	Incomplete demo	Tier system; Live first, Stretch last
Dataset license unclear	Can’t present on real data	Confirm Day 1; fall back to simglucose only
IAM / permissions friction	Lost hours debugging	Pre-build least-privilege roles Day 1

Risk	Impact	Mitigation
AI provider undecided	Rework on Day 5	Confirm Bedrock vs watsonx before Day 5
Real-time path flakiness	Weak demo moment	Pre-recorded fallback clip

11 Indicative Cost Posture

Cost control

The design is serverless-first (Lambda, S3, Athena, DynamoDB on-demand, Kinesis single shard) specifically to stay within or near AWS Free Tier for a short-lived demo. RDS is the main standing cost; use the smallest instance and stop it when idle. A teardown script removes all resources after the presentation.

Assumption / Open Question

Confirm your AWS account type (personal, student, or employer-provided). This affects free-tier eligibility and whether a budget cap is mandatory.

12 Consolidated Open Questions for Your Review

1. **AI provider:** Amazon Bedrock (AWS-native) or IBM watsonx (workshop alignment)?
2. **IBM tool:** Which exact tool from your workshop? (watsonx Orchestrate, watsonx.ai Studio, ContextForge, Langflow?)
3. **Dataset license:** Is the Shanghai figshare license acceptable for a work presentation?
4. **AWS account:** Personal / student / employer? Budget cap required?
5. **Time budget:** Confirm 5–7 hrs/day for 7 days, solo.
6. **Stretch items:** Keep USDA API and portability IaC, or cut to protect the core?
7. **Billing-code claim:** Are you comfortable including the US RPM billing-code framing, pending research confirmation?

13 Glossary

Term	Meaning
CGM	Continuous Glucose Monitoring — wearable sensor sampling blood glucose.
RPM	Remote Patient Monitoring — clinical program reviewing patient data remotely.
TIR/TBR/TAR	Time in / below / above Range — core CGM metrics.
CV	Coefficient of Variation — glucose variability measure.
FHIR	Fast Healthcare Interoperability Resources — healthcare data standard.

Term	Meaning
MCP	Model Context Protocol — standard for exposing tools to AI agents.
Medallion	Bronze/Silver/Gold layered data-quality pattern.
Iceberg	Open table format supporting ACID + time travel on object storage.
OLTP/OLAP	Transactional vs analytical processing.

End of Draft — Version 0.1

Please mark up this document and answer the seven open questions in Section 12. Once confirmed, I'll lock the scope and we start Day 1.